

# Learning-Based On-Track System Identification for Scaled Autonomous Racing in Under a Minute

Onur Dikici\*, Edoardo Ghignone<sup>†</sup>, Cheng Hu<sup>‡</sup>, Nicolas Baumann<sup>†</sup>,  
Lei Xie<sup>‡</sup>, Andrea Carron<sup>†</sup>, Michele Magno<sup>†</sup>, and Matteo Corno\*

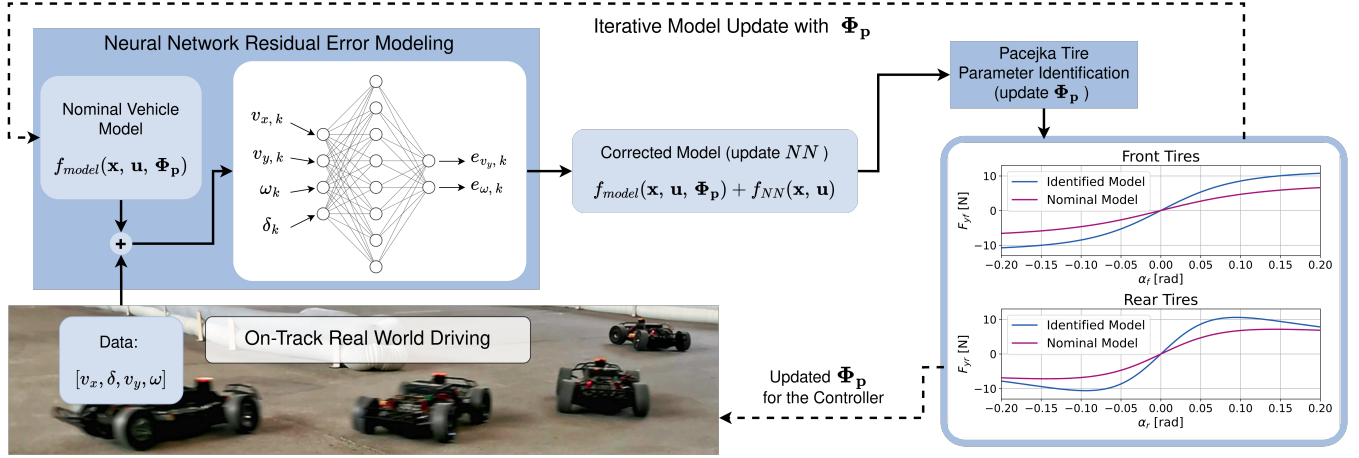


Figure 1: Schematic representation of the learning-based on-track system identification method for autonomous racing. The nominal vehicle model is corrected using a residual Neural Network (NN) error model, trained on data collected during on-track, real-world driving. Pacejka parameters are iteratively updated to enhance the overall accuracy of the vehicle model.

**Abstract**—Accurate tire modeling is crucial for optimizing autonomous racing vehicles, as State-of-the-Art (SotA) model-based techniques rely on precise knowledge of the vehicle’s parameters, yet system identification in dynamic racing conditions is challenging due to varying track and tire conditions. Traditional methods require extensive operational ranges, often impractical in racing scenarios. Machine Learning (ML)-based methods, while improving performance, struggle with generalization and depend on accurate initialization. This paper introduces a novel on-track system identification algorithm, incorporating a NN for error correction, which is then employed for traditional system identification with virtually generated data. Crucially, the process is iteratively reapplied, with tire parameters updated at each cycle, leading to notable improvements in accuracy in tests on a scaled vehicle. Experiments show that it is possible to learn a tire model without prior knowledge with only 30 seconds of driving data, and 3 seconds of training time. This method demonstrates greater one-step prediction accuracy than the baseline Nonlinear Least Squares (NLS) method under noisy conditions, achieving a 3.3x lower Root Mean Square Error (RMSE), and yields tire models with comparable accuracy to traditional steady-state system identification. Furthermore, unlike steady-state methods requiring large spaces and specific experimental setups, the proposed approach identifies tire parameters directly on a race track in dynamic racing environments.

**Index Terms**—Field Robots, Wheeled Robots, Machine Learning for Robot Control

\*Onur Dikici and Matteo Corno are associated with the Dipartimento di Elettronica, Informazione e Bioingegneria, Politecnico di Milano

<sup>†</sup>Edoardo Ghignone, Nicolas Baumann, Andrea Carron, and Michele Magno are associated with ETH Zürich.

<sup>‡</sup>Cheng Hu and Lei Xie are associated with the Department of Control Science and Engineering, Zhejiang University.

Onur Dikici, Edoardo Ghignone contributed equally to this work. (Corresponding author: Onur Dikici.)

## I. INTRODUCTION

Autonomous racing offers a dynamic, high-speed environment that requires high performance under extreme conditions, making it ideal for testing the limits of Autonomous Driving Systems (ADS) in scenarios similar to potential edge cases in real-world driving [1]. It provides a controlled yet demanding setting for advancing autonomous driving technologies without risking public traffic safety [2]. The challenging nature of racing drives the development of advanced data-driven algorithms that perform reliably at high speeds and handle complex dynamics, such as operating at the edge of friction. Additionally, racing supplies measurable metrics like lap time and tracking error to evaluate the performance of an ADS, addressing the difficulties of quantifying performance in general autonomous driving [2].

A primary challenge in autonomous racing is to complete a racetrack at high speeds [1, 2], which poses a demanding task for autonomous control systems due to the highly non-linear behavior of the car, when tire dynamics exceed the linear operating range and the vehicle must operate at the edge of traction [3–6]. Thus, accurately tracking trajectories at cornering speeds that saturate tire friction capacity is a central and highly challenging control task in autonomous racing [2, 7, 8].

To achieve said level of high performance, model-based controllers must utilize knowledge of vehicle model dynamics to compute effective control signals [4, 9]. These types of controllers require prior knowledge of tire dynamics, which are notoriously difficult to estimate in field conditions [4]. Thus, accurate system identification of the entire model, with

a particular focus on the complex tire dynamics, is essential for enabling high-performance model-based controllers [6].

Typically, this type of system identification is performed prior to a race through meticulous tire characterization experiments. These experiments are time-consuming and require a large, open space with a surface similar to the actual racetrack [4, 10, 11]. Enabling model-based controllers in real-world racing requires accurate tire parameter identification and robustness to variables like surface traction changes. System identification must be simple and deployable without large open spaces, which are often unavailable before a race [1]. Autonomous driving literature has identified two primary types of system identification methods:

**I Off-Track Identification:** These methods require steady-state conditions. They are accurate because system identification is conducted through steady-state experiments, providing a simplified dynamics environment with ideal conditions [4, 10–12]. Additionally, because the vehicle is in a steady-state, there is no need to deal with differentiating noisy measurements. However, they necessitate large, open spaces similar to the actual racetrack surface, which may not always be available, especially given the variable conditions within the nature of racing.

**II On-Track Identification:** These methods do not require large spaces for steady-state conditions but are more challenging due to the vehicle not being in a steady-state. Optimization methods, such as NLS [13, 14], are used to perform this type of identification but can be brittle and fail to produce reasonable results in practice. Different optimization methods can then successfully produce estimates, such as the non-convex program presented in [15] or the method in [16], which needs a good initial estimate of the tire parameters.

To address the limitations of previous system identification methods, this paper proposes a novel learning-based on-track system identification method. This method not only improves the accuracy of parameter identification but also significantly enhances the operational simplicity of tire parameter characterizations. It eliminates the need for space- and time-consuming identification maneuvers and overcomes the brittleness of conventional NLS-based on-track identification methods by leveraging the efficiency of data-driven NN learning. The contributions of this work are summarized as follows:

**I On-Track Identification:** The proposed method identifies tire characteristics directly on the track, eliminating the need for steady-state identification experiments in large open spaces [10]. This approach allows the identification process to be conducted on the actual track being used, removing the logistical challenges of finding a similar surface prior to the race and providing high operational simplicity. Furthermore, thanks to the combined method, the typical overfitting problem of ML techniques is avoided, as tire parameters are eventually obtained from our technique.

**II Robustness:** Our combined method using a NN-based learning scheme with traditional system identification demonstrates high robustness to noise encountered in

real-life deployments. Compared to NLS-based on-track identification methods, our approach achieves up to 3.3 times greater accuracy under noisy conditions in simulation and handles the noise experienced in real-world robotic systems, whereas conventional methods often fail to identify parameters outside of simulation environments.

**III Open-Source:** The proposed identification method is fully integrated into an open-source full-stack implementation [1], enhancing reproducibility and extensibility. Code available at: <https://github.com/ForzaETH/On-Track-SysID>.

## II. RELATED WORK

In system identification for autonomous vehicles, steady-state techniques like [10, 11] face limitations due to their requirement for large free areas for circular motion. The work presented in [11] uses high-fidelity simulators and high-quality initial estimates, while other approaches rely on provided ground truth tire data to learn [12]. On-track methods [6, 14–17], which utilize data collected during driving along the same track destined for a competition, present a more practical alternative. Among these methods, classical approaches like NLS are some of the suggested ones [13], and they are implemented online for instance in [14]. However, these methods could be sensitive to noisy data, and [17] shows that simple data-driven techniques can outperform least-squares-based system identification on real data.

Different approaches employed Gaussian Processes (GPs) for efficiently exploiting real data, such as [18], where a kinematic model is extended with a GP and iteratively updates to learn model mismatch in simulation, or [19] where the authors present a method to utilize an ensemble of GPs to be able to model varying friction. Both these models however suffer from computational limitations, especially with the need to continuously update the models, and, to mitigate computational costs, different works utilize sparse GP regression, such as [16, 20]. Despite these improvements, their methods remain untested under significant model mismatches, as they often start with an initially accurate physical model.

Furthermore, NN techniques have been proposed to address the computational cost, such as in [21] where an Extreme Learning Machine (ELM) is used to successfully identify dynamic vehicle models in simulation, or in [22] where an end-to-end NN is proposed for learning vehicle dynamics. This last work, however, highlights how these ML techniques can fail to generalize across diverse conditions and would require extensive data gathering to be deployed in online applications. Recent work explored the adaptability with ML models more extensively, such as with Continual-MAML [23], or with different architectures such as LSTMs [24] or Transformers [25].

This work builds on top of this last direction, aiming to exploit the efficiency in data processing of NNs and to address two main issues with the hybrid architecture joining ML and traditional system identification. Firstly, we show how our method can be more practical than end-to-end learning techniques [21, 22] and steady-state system identification

procedures [10, 11] by learning with less than a minute of training data and not requiring the design of specific experiments or significantly large spaces. Furthermore, we address the need for adaptable models, already highlighted by [14, 19], by presenting a lightweight model that can adapt to new data in at most 3 seconds of training time, effectively enabling online adaptation on a fully onboard setup. Crucially, our method differs from adaptive ML techniques for learning dynamic models [23–25] in a key way: rather than only updating the NN, our approach iteratively updates both the tire model and the NN. Jointly updating both models allows for accurate dynamic representation even with a relatively simple model comprising less than 60 parameters. Furthermore, it also paves the way for future work intersecting novel techniques [23–25] and traditional empirical methods such as the Pacejka tire model.

### III. SYSTEM IDENTIFICATION

In this section, we provide an overview of our methodology, which proposes a NN model to capture the model mismatch. Although modeling the mismatch with a data-driven approach may fail to represent out-of-distribution data, simulating and generating predictions with in-distribution data enables the extraction of the underlying Pacejka parameters embedded within the combined NN and vehicle model. Furthermore, this process is done iteratively to gradually improve the accuracy of the identified tire model with each iteration.

#### A. Vehicle Model

The vehicle model used in this work illustrated in Figure 2, is the dynamic single-track model, a commonly used vehicle model in the field of autonomous racing as it effectively models tire-road interactions in steady-state conditions [26]. This model simplifies the vehicle to one wheel per axis and decouples longitudinal and lateral dynamics. While this approach captures the essential vehicle behavior, it relies on a steady-state tire model that may not fully account for the dynamic response of tires during rapid maneuvers. The vehicle is considered to be a rigid body with a mass  $m$  and an inertia  $I_z$  around the z-axis of the center of gravity.  $l_f$  and  $l_r$  represent the distance of the front and rear axle from the center of gravity, respectively.

The equation of motion for the lateral model can be expressed as follows:

$$\begin{aligned} \dot{v}_y &= \frac{1}{m} (F_{yr} + F_{yf} \cos \delta - m v_x \omega) \\ \dot{\omega} &= \frac{1}{I_z} (F_{yf} l_f \cos \delta - F_{yr} l_r). \end{aligned} \quad (1)$$

To accurately model the lateral forces  $F_{yf}$  and  $F_{yr}$ , the Pacejka *Magic Formula* [3] is used in its general form

$$F_{yi} = D_i \sin(C_i \arctan(B_i \alpha_i - E_i(B_i \alpha_i - \arctan(B_i \alpha_i)))). \quad (2)$$

where  $i$  is an element of  $\{f, r\}$ , representing the front and rear tires respectively. Due to the low center of gravity of the scaled car used in this work, longitudinal and lateral load

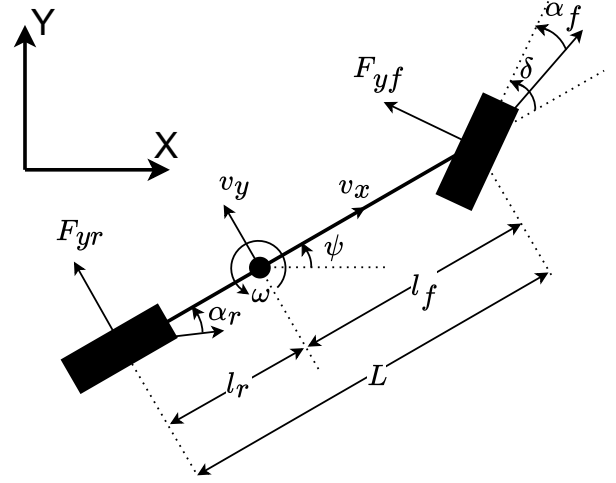


Figure 2: Dynamic single-track model showing lateral forces ( $F_{yf}$ ,  $F_{yr}$ ), slip angles ( $\alpha_f$ ,  $\alpha_r$ ), velocities ( $v_x$ ,  $v_y$ ), steering angle ( $\delta$ ), yaw rate ( $\omega$ ), angle with respect to the positive X axis ( $\psi$ ), and X-Y coordinate system. The symbols  $l_r$ ,  $l_f$  indicate, respectively, the distance from the center of gravity to the rear and front axle, and  $L$  indicates the full wheelbase length. *Note: This image is illustrative; slip angles are exaggerated for visualization purposes.*

transfers are neglected. The slip angles are also computed as follows:

$$\begin{aligned} \alpha_f &= \delta - \arctan\left(\frac{v_y + l_f \omega}{v_x}\right) \\ \alpha_r &= -\arctan\left(\frac{v_y - l_r \omega}{v_x}\right) \end{aligned} \quad (3)$$

This work focuses solely on lateral vehicle dynamics, specifically the identification of Pacejka model parameters  $\Phi_p = [B_f, C_f, D_f, E_f, B_r, C_r, D_r, E_r]$ . Accordingly, we treat  $\mathbf{x} = [v_y, \omega]$  as states, and  $\mathbf{u} = [v_x, \delta]$  as inputs. Using Euler integration, the discrete-time vehicle model becomes:

$$\begin{aligned} v_{y,k+1} &= v_{y,k} + \frac{1}{m} (F_{yr} + F_{yf} \cos \delta - m v_x \omega) T_s \\ \omega_{k+1} &= \omega_k + \frac{1}{I_z} (F_{yf} l_f \cos \delta - F_{yr} l_r) T_s, \end{aligned} \quad (4)$$

where  $T_s$  is the sampling time.

#### B. Model Mismatch Problem

There are instances when the identified Pacejka parameters  $\Phi_p$  may no longer be accurate. This can result from changes in the racing surface—as Pacejka parameters are surface-dependent—as well as from tire wear and variations in tire temperature. Consequently, there may be a discrepancy between the predicted subsequent states  $\hat{\mathbf{x}}_{k+1}$  and the measured subsequent states  $\mathbf{x}_{k+1}$ . This discrepancy is known as model mismatch and is expressed as follows:

$$\mathbf{e}_k = \mathbf{x}_{k+1} - \hat{\mathbf{x}}_{k+1}, \quad (5)$$

where  $\hat{\mathbf{x}}_{k+1} = [\hat{v}_{y,k+1}, \hat{\omega}_{k+1}]$  is calculated using (4) with the nominal Pacejka parameters  $\Phi_p$ .

### C. Data Collection & Data Processing

1) **Data Collection:** Data is collected by driving the vehicle on a track with a Pure Pursuit controller [27], using the implementation from [1]. Crucially, such a controller does not require any tire or vehicle model knowledge apart from the wheelbase length  $L$ , making it apt as a starting baseline. A single collected data tuple for NN training is identified with  $\mathcal{D}^{train} = [v_x, v_y, \omega, \delta]$ .

2) **Data Processing:** After collecting the data, preprocessing is performed to prepare it for NN training. First, a noncausal low-pass filter is applied to handle noise from measurement errors and environmental factors, ensuring smooth and reliable data without introducing phase delays. Second, assuming symmetric vehicle dynamics, the dataset is augmented by mirroring lateral velocity, yaw rate, and steering angle, while keeping longitudinal velocity unchanged. This augmentation increases the data volume and eliminates the cornering bias inherent to racing tracks.

### D. Neural Network for Identification

The NN is tasked with learning and correcting the model error to improve the nominal vehicle model. The targets of the NN are the prediction errors  $\mathbf{e}_k = [e_{v_{y,k}}, e_{\omega_k}]$  for the subsequent step's lateral velocity and yaw rate. These errors are determined as described in (5).

The inputs for the NN are chosen to be  $[\mathbf{x}_k, \mathbf{u}_k] = [v_{x,k}, v_{y,k}, \omega_k, \delta_k]$  since the model error depends on these variables in the determined vehicle model. In summary, the NN models the prediction error of a nominal vehicle model based on these inputs. This function learned by the NN is denoted as follows:

$$\mathbf{e}_k = f_{NN}(v_{x,k}, v_{y,k}, \omega_k, \delta_k), \quad (6)$$

representing the prediction of the error  $\mathbf{e}_k$  given the inputs.

### E. Parameter Identification with the Corrected Vehicle Model

The NNs performance is constrained by the limited training data available, making globally accurate behavior challenging and struggling with sparse, non-existent, or significantly different inputs from the training data (out-of-distribution). To address those limitations, we identify Pacejka parameters, as the Pacejka tire model accurately represents vehicle dynamics across diverse conditions. By simulating the corrected vehicle model with a range of well-represented input variables, we ensure that the NN operates within in-distribution data, leading to reliable predictions.

1) **The Corrected Vehicle Model:** The corrected vehicle model leverages both the NN and the nominal vehicle model, similarly to [5]. The nominal model provides a baseline prediction, while the NN corrects residual errors. This structure is shown in Figure 3.

The corrected next states  $[v_{y,k+1}, \omega_{k+1}]$  are calculated as follows:

$$\mathbf{x}_{k+1} = \hat{\mathbf{x}}_{k+1} + \mathbf{e}_k, \quad (7)$$

where  $\hat{\mathbf{x}}_{k+1}$  is the predicted next state from the nominal vehicle model using (4), and  $\mathbf{e}_k$  is the residual error predicted by the NN using (6).

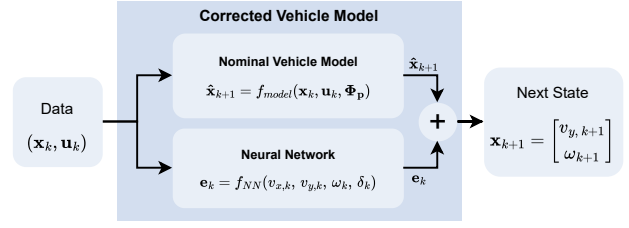


Figure 3: The corrected vehicle model structure combining a nominal vehicle model and a NN to predict the next state variables  $v_{y,k+1}$  and  $\omega_{k+1}$ . Where  $f_{\text{model}}(\mathbf{x}_k, \mathbf{u}_k, \Phi_p)$  represents the state transition function based on (4), with  $\mathbf{x}_k$  and  $\mathbf{u}_k$  being the state and input vectors, respectively, and  $\Phi_p$  are the Pacejka parameters.

2) **Virtual Steady-State Data Generation:** To identify Pacejka parameters, we first generate virtual steady-state data by simulating the corrected vehicle model at a constant longitudinal speed. The chosen speed is the average longitudinal velocity of the training data, ensuring the NN performs confidently due to its familiarity with this type of data. During the simulation, the steering angle is gradually increased from 0 to 0.4 radians linearly, based on the operational limits of the target platform, a 1:10 scaled vehicle. By incrementing the steering angle in this controlled manner, we can observe and record the vehicle's response under steady-state conditions.

The simulation initializes the states  $\mathbf{x}_0 = [v_{y,0}, \omega_0] = [0, 0]$ . The prior next state predictions,  $\hat{\mathbf{x}}_{k+1}$ , from the nominal model are corrected with the NN's error prediction,  $\mathbf{e}_k$ , using the current states and inputs as described in (7). This process generates the entire steady-state data over 10 seconds with a time step of 0.02 seconds.

3) **Pacejka Model Parameters Identification:** Since the generated data is in steady-state, we assume  $\dot{v}_y = 0$  and  $\dot{\omega} = 0$ . This allows us to accurately compute the lateral forces  $F_{yr}$  and  $F_{yf}$  using the states and inputs:

$$\begin{aligned} F_{yr} &= \frac{ml_f}{l_f + l_r} v_x \omega \\ F_{yf} &= \frac{ml_r}{l_f + l_r} \frac{v_x \omega}{\cos(\delta)} \end{aligned} \quad (8)$$

The slip angles  $\alpha_f$  and  $\alpha_r$  are calculated using (3). We then fit the Pacejka model to the force vs. slip angle data  $(F_{yf}, \alpha_f)$  and  $(F_{yr}, \alpha_r)$  through least squares regression [4, 10].

### F. Iterative Learning

Even though the identified Pacejka parameters improve the model, a mismatch may still persist if the initial mismatch is high. To address this, we iteratively refine the model by using the identified parameters as a new nominal model, learning its residuals, and repeating the process until the model converges. The convergence is determined empirically by observing that further iterations do not yield significant improvements in the identified Pacejka models. This approach ensures that the NN continuously learns residual errors based on progressively more accurate Pacejka parameters, improving the overall vehicle model accuracy. The scheme for this iterative approach is demonstrated in Figure 4.

One significant advantage of this iterative approach is that it removes the dependency on having an initially accurate nominal model. Even if the initial model has a high mismatch, the iterative process allows for continuous improvement of the nominal model's accuracy until it converges. By identifying Pacejka parameters and re-initializing the NN, we prevent overfitting and progressively refine the model's accuracy.

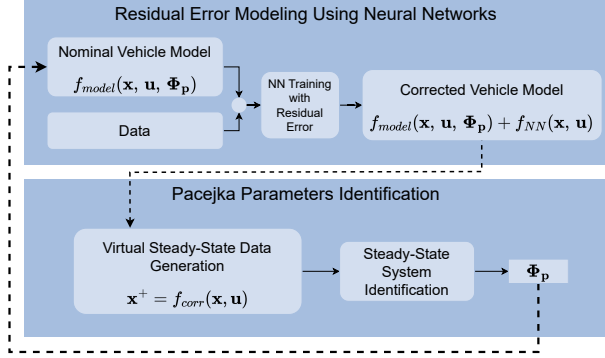


Figure 4: Scheme of the iterative approach: Starting with residual error modeling and corrected vehicle model formation, followed by steady-state system identification using virtual data from the corrected vehicle model. Identified parameters are then used as a new nominal model for learning residual errors, repeating iteratively until convergence.

#### IV. EXPERIMENTAL RESULTS

In this section, we present the metrics utilized for assessing the accuracy and performance of the identified models and then show the results in simulation and on the physical 1:10 scaled autonomous racing platform.

##### A. Model Architecture

The NN architecture used is a feed-forward Multilayer Perceptron (MLP) with an input layer of 4 neurons, one hidden layer of 8 neurons, and an output layer of 2 neurons, comprising a total of 58 parameters. Due to our limited dataset size, we chose a simple model to reduce the risk of overfitting. Initial tests showed that adding layers or neurons did not improve accuracy and instead led to overfitting, capturing noise over useful patterns. This simpler structure provided computational efficiency while effectively capturing essential data patterns. Both the input and hidden layers use the LeakyReLU activation function to introduce non-linearity, while the output layer uses a linear activation function to produce the final predictions. For training, we use the Adam optimizer with a learning rate of  $5 \times 10^{-4}$ , determined through systematic testing of values ranging from  $10^{-2}$  to  $10^{-6}$ . This learning rate provided stable convergence behavior and minimized training time without sacrificing model performance. The loss function used is Mean Square Error (MSE). We use a batch size equal to the entire dataset size, performing batch gradient descent for each training step. This approach reduces the impact of noisy data points and is feasible given the limited size of our dataset.

##### B. Model Validation and Performance Assessment Metrics

1) *One-Step Prediction Errors*: At each time step in the test set  $\mathcal{D}^{test}$ , generated from a different run on the same track, the identified models generate predictions for the subsequent states,  $\hat{\mathbf{x}}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k)$ . These predictions are then compared with the observed subsequent states,  $\mathbf{x}_{k+1}$ , using the RMSE to evaluate prediction accuracy.

2) *Racing Performance*: We will also evaluate the lap time to complete a lap as well as the average and maximum lateral deviation from the racing trajectory achieved by a model-based lateral controller [4] utilizing the identified model parameters. These metrics serve as a practical performance measure in a real-world scenario, indicating how well the model supports the controller in achieving high racing performance.

##### C. Simulation Results

1) *Experimental Setup*: The simulation experiments are conducted in the F1TENTH simulator [1], which was adapted to use the single-track vehicle model presented in Section III-A with parameters similar to those of the real vehicle to minimize the sim-to-real gap. The simulator also provides ground truth data.

To benchmark our approach, we also employ the NLS method for comparison, applying the same low-pass filtering to the data for both approaches to ensure a fair comparison. The objective function for NLS aims to find the best Pacejka parameters  $\Phi_p$  that minimize the sum of squared residuals between the observed and predicted states over all time steps and is formulated as follows:

$$\min_{\Phi_p} \sum_{k=1}^N \alpha_{v_y}^2 (v_{y,k+1} - \hat{v}_{y,k+1})^2 + \alpha_{\omega}^2 (\omega_{k+1} - \hat{\omega}_{k+1})^2. \quad (9)$$

The scaling parameters  $\alpha_{v_y}$ ,  $\alpha_{\omega}$  were both set to 1 throughout this work. For both approaches, 30 seconds of driving data, corresponding to 2 laps on the track, with a time step of 0.02 seconds is collected within the simulator by driving the vehicle with a model-free Pure Pursuit [27] controller to follow the predefined trajectory.

Given the simulator's simplified vehicle model and the availability of ground truth data, both methods are expected to yield accurate models. To evaluate the robustness of each method under noisy conditions, we introduce artificial Gaussian noise to the data, simulating real-world measurement noise. The procedure for adding noise is as follows:

$$\mathbf{d}_{noisy} = \mathbf{d} + \mathcal{N}(0, \mathbf{d}_{avg} \cdot \eta), \quad (10)$$

where  $\mathbf{d} = [v_x, v_y, \omega, \delta]$  represents the state and input variables,  $\mathbf{d}_{avg}$  are their respective average magnitudes to maintain a consistent noise-to-signal ratio across different levels of the state and input variables.  $\mathcal{N}(0, \sigma)$  denotes Gaussian noise with zero mean and standard deviation  $\sigma$ , scaled by the noise multiplier  $\eta$ , which is increased gradually after each experiment from 0 to 1.4 with an increment of 0.2.

2) *Model Validation Testing*: Figure 5 illustrates the RMSE of lateral velocity ( $v_y$ ) and yaw rate ( $\omega$ ) predictions under varying levels of noise. The x-axis represents the noise



standard deviation multiplier ( $\eta$ ), which scales the standard deviation of the Gaussian noise added to the data.

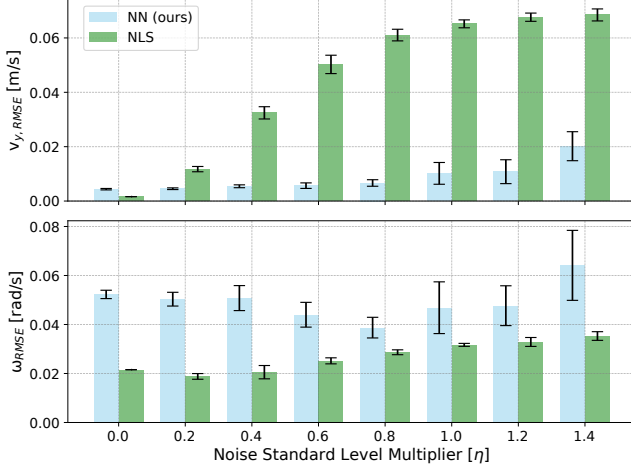


Figure 5: The RMSE of lateral velocity and yaw rate predictions for the models identified using our approach and the NLS on the test set under increasing levels of noise. Experiments were repeated 10 times.

As our initial tests showed that the identified Pacejka models typically converge within six iterations across all noise levels, we run our approach for six iterations. For lateral velocity ( $v_y$ ), our approach consistently outperforms the NLS method across all noise levels ( $\eta$ ). In terms of yaw rate ( $\omega$ ), the performance of our approach is comparable to that of the NLS method. When averaging the performance across both  $v_y$  and  $\omega$ , the proposed method achieves an RMSE that is 3.3 times lower for the one-step prediction compared to the NLS method, highlighting its robustness and accuracy in handling noisy conditions.

#### D. Physical FITENTH Results

1) *Experimental Setup*: For the real-world experiments, we used an FITENTH vehicle as detailed in [1]. This setup includes an onboard Intel NUC computer running all software algorithms within the ROS framework, enabling indoor localization and state estimation, as in [1].

In practice, due to the absence of a ground truth model, we use the widely adopted steady-state system identification [10], as a baseline. This method is expected to yield a highly accurate model because the Pacejka parameters are identified under controlled, simplified steady-state conditions. Additionally, the NLS method, with the objective function (9), is used as another on-track system identification benchmark to compare with the proposed approach.

2) *Data Collection*: For steady-state identification, three experiments were conducted at constant velocities ( $2\text{--}3\text{ m s}^{-1}$ ) and ramp steering angles ( $0.2\text{--}0.4$  radians), covering a range of slip angles in a controlled  $6.5\text{ m} \times 6.5\text{ m}$  space.

For both the NLS and our approach, 30 seconds of driving data—equivalent to approximately 3 laps—was collected on the same surface used for steady-state experiments. During these trials, the vehicle was driven to follow a predefined

trajectory using a model-free Pure Pursuit [27] controller, assuming no prior knowledge of tire parameters. This method provided diverse and dynamic data, crucial for on-track system identification. The choice of 30 seconds was based on preliminary tests, which showed it was sufficient for robust parameter identification.

3) *Model Validation Testing*: Figure 6 shows the identified Pacejka models for both front and rear tires using the three approaches. The models identified with our approach align well with the baseline steady-state method, while the NLS fails to produce similar models. At higher lateral slip angles—greater than  $0.15$  and  $0.13$  radians for the front and rear tires, respectively—the models diverge due to insufficient data in that region, as it lies outside the operational region. Therefore, our region of interest is limited to slip angles below  $0.15$  and  $0.13$  radians for the front and rear tires, respectively. Within this region, both our approach and the steady-state method show perfect alignment for the front tires, while slight deviations for the rear tires are observed, likely due to noise in the data.

Table I presents a comparison of the three methods using key metrics, including the RMSE for lateral velocity ( $v_y$ ) and yaw rate ( $\omega$ ) on the test data, as well as lap time, average lateral error, and maximum lateral error. All runs were performed with a Model- and Acceleration-based Pursuit (MAP) controller [1, 4], utilizing the tire parameters identified by each method. Despite being considered the gold standard, the steady-state method shows the worst RMSE performance for  $\omega$  on a noisy real-world dataset, as shown in Table I. Similarly, although the NLS achieves the lowest RMSE for  $\omega$ , the resulting Pacejka parameters underestimate tire forces, as shown in Figure 6, leading to undrivable (i.e., N.C.) behavior in closed-loop tests.

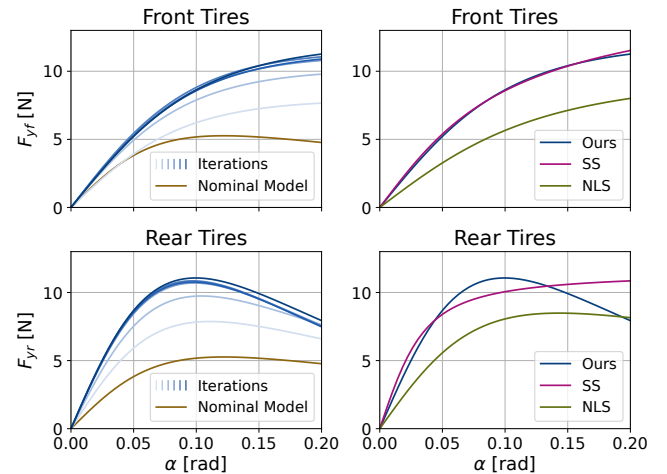


Figure 6: The left plots illustrate the evolution of the resultant Pacejka models using our method over six iterations, starting from the nominal model (bronze) and transitioning through shades of blue (light to dark). The right plots show the identified Pacejka models using three methods: Ours, Steady-State (SS), and NLS.

| Metrics                 | Ours          | Steady-State | NLS           |
|-------------------------|---------------|--------------|---------------|
| $v_{y, RMSE}$ [m/s]     | <b>0.0233</b> | 0.0239       | 0.0289        |
| $\omega_{RMSE}$ [rad/s] | 0.106         | 0.108        | <b>0.0655</b> |
| $t_{lap}^\mu$ [s]       | <b>8.10</b>   | 8.16         | N.C.          |
| $ d^\mu $ [m]           | <b>0.082</b>  | 0.092        | N.C.          |
| $ d^{max} $ [m]         | <b>0.27</b>   | 0.3          | N.C.          |

Table I: The RMSE values for lateral velocity  $v_y$  and yaw rate  $\omega$ , along with lap times, average, and maximum deviation from the trajectory for different methods: our approach, the steady-state system identification method, and the NLS approach. N.C.: Not completed a lap without track violations

The experimental results demonstrate that our proposed on-track system identification method accurately identifies tire parameters, producing Pacejka models similar to those obtained from traditional steady-state methods, particularly within the critical slip angle range. Our approach offers significant operational simplicity compared to steady-state system identification and greater robustness compared to the classical NLS method while achieving the best racing performance among the benchmark methods.

Most notably, our approach requires only six iterations, with each iteration taking approximately 0.5 seconds, totaling just 3 seconds to learn the complete vehicle model, even without accurate prior knowledge. This confirms the accuracy, efficiency, and practicality of our method for real-world autonomous racing applications.

#### E. Dynamic Adaptation

This experiment evaluates our method’s ability to adapt from an initial hard tire model to a soft tire model under real-world conditions, such as tire softening due to temperature changes. A controller based on the hard tire model is employed while the vehicle operates with soft tires. Since only a slight model mismatch is assumed, our method is executed only for two iterations, completing in a total of 1 second. As shown in Table II, adapting to the soft tire model achieves a lap time of 8.5 seconds and an average lateral error of 0.11 meters, outperforming the initial hard tire model by 0.25 seconds and reducing the lateral error by 0.05 meters.

| Metrics           | Hard Tire Model | Adapted Soft Tire Model |
|-------------------|-----------------|-------------------------|
| $t_{lap}^\mu$ [s] | 8.75            | <b>8.5</b>              |
| $ d^\mu $ [m]     | 0.16            | <b>0.11</b>             |

Table II: Performance comparison showing adaptation from an initial hard tire model to a soft tire model, reflecting real-time responsiveness to changes in tire conditions.

These results demonstrate our method’s ability to adapt in real-time within just 1 second, utilizing 30 seconds of driving data, making it suitable for dynamic racing conditions that require rapid adjustments.

## V. CONCLUSIONS

This paper presents a learning-based on-track system identification method for autonomous racing, integrating a NN

with a nominal Pacejka tire model. The approach eliminates the need for large, open spaces such as required by traditional steady-state methods by utilizing on-track data, simplifying the process, and achieving up to 3.3 times lower RMSE under noise compared to standard NLS methods. It uses three times less data than steady-state methods, learning the model in 33 seconds, and adapts in 1 second for environmental changes, enabling real-time updates. The identified Pacejka models closely align with steady-state methods within critical slip angles, demonstrating its practicality, robustness, and efficiency for advancing autonomous racing performance. Different lines of future work are possible. One direction is to improve the accuracy of the nominal model by precisely refining the tire model and then combining it with more complex ML models, such as those presented in [24, 25]. Another direction involves exploring metrics beyond one-step prediction RMSE that could better assess the performance of the identified Pacejka models and help define a stopping criterion, as the current metric has proven to be not entirely indicative of Pacejka parameters identification performance. Furthermore, given its small computational footprint, the full NN corrected vehicle model could be used in the context of a simulator, similarly to [6], or in a sampling-based Model Predictive Control (MPC) method as in [24]. Finally, further extensions could also consider applying the same algorithm to other robotic platforms, where system identification would require inconvenient infrastructure.

## ACKNOWLEDGMENT

The authors would like to thank all team members of the *ForzaETH* racing team. In particular Niklas Bastuck for the photography of the live car in Figure 1.

## REFERENCES

- [1] N. Baumann, E. Ghignone, J. Kühne, N. Bastuck, J. Becker, N. Imholz, T. Kränzlin, T. Y. Lim, M. Lötscher, L. Schwarzenbach *et al.*, “Forzaeth race stack-scaled autonomous head-to-head racing on fully commercial off-the-shelf hardware,” *arXiv preprint arXiv:2403.11784*, 2024.
- [2] J. Betz, H. Zheng, A. Liniger, U. Rosolia, P. Karle, M. Behl, V. Krovi, and R. Mangharam, “Autonomous vehicles on the edge: A survey on autonomous vehicle racing,” *IEEE Open Journal of Intelligent Transportation Systems*, vol. 3, pp. 458–488, 2022.
- [3] H. B. Pacejka and E. Bakker, “The magic formula tyre model,” *Vehicle system dynamics*, vol. 21, no. S1, pp. 1–18, 1992.
- [4] J. Becker, N. Imholz, L. Schwarzenbach, E. Ghignone, N. Baumann, and M. Magno, “Model-and acceleration-based pursuit controller for high-performance autonomous racing,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 5276–5283.
- [5] H. Xue, E. L. Zhu, J. M. Dolan, and F. Borrelli, “Learning model predictive control with error dynamics regression for autonomous racing,” in *2024 IEEE*

- International Conference on Robotics and Automation (ICRA)*, 2024, pp. 13 250–13 256.
- [6] J. Chrosniak, J. Ning, and M. Behl, “Deep dynamics: Vehicle dynamics modeling with a physics-constrained neural network for autonomous racing,” *IEEE Robotics and Automation Letters*, vol. 9, no. 6, pp. 5292–5297, 2024.
  - [7] J. Kabzan, M. I. Valls, V. J. F. Reijgwart, H. F. C. Hendriks, C. Ehmke, M. Prajapat, A. Bühler, N. Gosala, M. Gupta, R. Sivanesan, A. Dhall, E. Chisari, N. Karnchanachari, S. Brits, M. Dangel, I. Sa, R. Dubé, A. Gawel, M. Pfeiffer, A. Liniger, J. Lygeros, and R. Siegwart, “AMZ driverless: The full autonomous racing system,” *J. field robot.*, no. rob.21977, Aug. 2020.
  - [8] A. Wischniewski, M. Geisslinger, J. Betz, T. Betz, F. Fent, A. Heilmeyer, L. Hermansdorfer, T. Herrmann, S. Huch, P. Karle, F. Nobis, L. Ögretmen, M. Rowold, F. Sauerbeck, T. Stahl, R. Trauth, M. Lienkamp, and B. Lohmann, “Indy autonomous challenge – autonomous race cars at the handling limits,” 2022. [Online]. Available: <https://arxiv.org/abs/2202.03807>
  - [9] A. Liniger, A. Domahidi, and M. Morari, “Optimization-based autonomous racing of 1:43 scale rc cars,” *Optimal Control Applications and Methods*, vol. 36, no. 5, p. 628–647, Jul 2014. [Online]. Available: <http://dx.doi.org/10.1002/oca.2123>
  - [10] C. Voser, R. Y. Hindiyeh, and J. C. Gerdes, “Analysis and control of high sideslip manoeuvres,” *Vehicle System Dynamics*, vol. 48, no. sup1, pp. 317–336, 2010. [Online]. Available: <https://doi.org/10.1080/00423111003746140>
  - [11] A. Raji, A. Liniger, A. Giove, A. Toschi, N. Musiu, D. Morra, M. Verucchi, D. Caporale, and M. Bertogna, “Motion planning and control for multi vehicle autonomous racing at high speeds,” in *2022 IEEE 25th International Conference on Intelligent Transportation Systems (ITSC)*, 2022, pp. 2775–2782.
  - [12] H. Seong, C. Chung, and D. H. Shim, “Model parameter identification via a hyperparameter optimization scheme for autonomous racing systems,” *IEEE Control Systems Letters*, vol. 7, pp. 1652–1657, 2023.
  - [13] R. Rajamani, *Vehicle Dynamics and Control*, 01 2006.
  - [14] M. Brunner, U. Rosolia, J. Gonzales, and F. Borrelli, “Repetitive learning model predictive control: An autonomous racing example,” in *2017 IEEE 56th Annual Conference on Decision and Control (CDC)*, 2017, pp. 2545–2550.
  - [15] S. Bodmer, L. Vogel, S. Muntwiler, A. Hansson, T. Bodewig, J. Wahlen, M. N. Zeilinger, and A. Carron, “Optimization-based system identification and moving horizon estimation using low-cost sensors for a miniature car-like robot,” *arXiv preprint arXiv:2404.08362*, 2024.
  - [16] J. Kabzan, L. Hewing, A. Liniger, and M. N. Zeilinger, “Learning-based model predictive control for autonomous racing,” *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3363–3370, 2019.
  - [17] B. A. H. Vicente, S. S. James, and S. R. Anderson, “Linear system identification versus physical modeling of lateral-longitudinal vehicle dynamics,” *IEEE Transactions on Control Systems Technology*, vol. 29, no. 3, pp. 1380–1387, 2021.
  - [18] A. Jain, M. O’Kelly, P. Chaudhari, and M. Morari, “Bayesrace: Learning to race autonomously using prior experience,” in *Proceedings of the 2020 Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, J. Kober, F. Ramos, and C. Tomlin, Eds., vol. 155. PMLR, 16–18 Nov 2021, pp. 1918–1929. [Online]. Available: <https://proceedings.mlr.press/v155/jain21b.html>
  - [19] T. Nagy, A. Amine, T. X. Nghiem, U. Rosolia, Z. Zang, and R. Mangharam, “Ensemble gaussian processes for adaptive autonomous driving on multi-friction surfaces,” *IFAC-PapersOnLine*, vol. 56, no. 2, pp. 494–500, 2023, 22nd IFAC World Congress. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2405896323020244>
  - [20] L. Hewing, A. Liniger, and M. N. Zeilinger, “Cautious NMPC with gaussian process dynamics for miniature race cars,” *CoRR*, vol. abs/1711.06586, 2017. [Online]. Available: <http://arxiv.org/abs/1711.06586>
  - [21] D. Kalaria, Q. Lin, and J. M. Dolan, “Adaptive planning and control with time-varying tire models for autonomous racing using extreme learning machine,” 2023. [Online]. Available: <https://arxiv.org/abs/2303.08235>
  - [22] L. Hermansdorfer, R. Trauth, J. Betz, and M. Lienkamp, “End-to-end neural network for vehicle dynamics modeling,” in *2020 6th IEEE Congress on Information Science and Technology (CiSt)*, 2020, pp. 407–412.
  - [23] Y. Tsuchiya, T. Balch, P. Drews, and G. Rosman, “Online adaptation of learned vehicle dynamics model with meta-learning approach,” 2024. [Online]. Available: <https://arxiv.org/abs/2409.14950>
  - [24] D. Kalaria, H. Xue, W. Xiao, T. Tao, G. Shi, and J. M. Dolan, “Agile mobility with rapid online adaptation via meta-learning and uncertainty-aware mppi,” 2024. [Online]. Available: <https://arxiv.org/abs/2410.06565>
  - [25] W. Xiao, H. Xue, T. Tao, D. Kalaria, J. M. Dolan, and G. Shi, “Anycar to anywhere: Learning universal dynamics model for agile and adaptive mobility,” 2024. [Online]. Available: <https://arxiv.org/abs/2409.15783>
  - [26] T. Zhang, Y. Sun, Y. Wang, B. Li, Y. Tian, and F.-Y. Wang, “A survey of vehicle dynamics modeling methods for autonomous racing: Theoretical models, physical/virtual platforms, and perspectives,” *IEEE Transactions on Intelligent Vehicles*, vol. 9, no. 3, pp. 4312–4334, 2024.
  - [27] J. M. Snider, “Automatic steering methods for autonomous automobile path tracking,” 2009. [Online]. Available: <https://api.semanticscholar.org/CorpusID:17512121>